The Open Source Audit

A Capstone Project for OSS NGMC Course

Field	Details
Name	Md. Ausaaf Khan
Registration Number	24MIP10107
Slot	E11
Software Chosen	Python
OS Used	Ubuntu 25.10
Kernal	6.17.0-5-generic
Date of Submission	30 March 2026

Introduction

Open source software plays an important role in today's technology. It allows users to access, modify, and share software freely. Unlike proprietary software, open source encourages collaboration and transparency.

In this project, I have selected Python to study its origin, license, and role in the open-source ecosystem. I chose Python because it is one of the most widely used languages today and almost everything around it the language itself, its tools, its libraries is built and maintained by the community. Along with this, I have also created five shell scripts on a Linux system to understand how open-source tools work practically.

While working on this project, I used Ubuntu 25.10 running inside a UTM virtual machine on macOS and explored different Linux commands and scripting concepts. Setting up the environment and running the scripts myself helped me understand things that I wouldn't have learned just by reading about them.

Part A : Origin and Philosophy

A1. The Problem Python Was Created to Solve

Python was created to solve the problem of complexity in programming. Before Python, most popular programming languages like C and C++ were powerful but difficult to use, especially for beginners. Writing even small programs required a lot of code, and understanding the syntax was not easy.

At that time, programming was mainly accessible to experienced developers because the languages were not very readable. This created a barrier for new learners who wanted to start coding. Even for professionals, development could be slow because writing and debugging complex code took a lot of time.

Guido van Rossum noticed this issue and wanted to create a language that was simpler and more readable. He was working at CWI in the Netherlands in the late 1980s and started writing Python mostly as a side project during the Christmas holidays of 1989 not because a company asked him to, but because he genuinely felt something better

was possible. His goal was to reduce the difficulty of programming and make it more accessible to a wider group of people. He focused on creating a language where the code looks clean and easy to understand, so that developers can spend more time solving problems instead of struggling with syntax.

Another problem was productivity. Existing languages required more lines of code to perform tasks, which increased development time. Python was designed to allow developers to write shorter and more efficient code, making the development process faster. When Guido released Python 0.9.0 in 1991, he shared it publicly on a Usenet newsgroup and invited feedback. He wasn't trying to build something famous he just wanted people to try it and tell him what they thought. What happened instead was that a whole community formed around it, because a lot of programmers had been quietly waiting for exactly this kind of language.

By solving these problems, Python made programming easier to learn and more efficient to use. It helped beginners enter the field of programming and allowed developers to build applications more quickly.

While working on this project, I understood how important readability and simplicity are in programming. When I ran my own scripts on Ubuntu and compared how much I could do in just a few lines, it made sense why Python grew so fast. It clearly solves the problem of complexity that existed in earlier programming languages, and the fact that Guido shared it openly from day one is a big reason why it became what it is today.

A1. The Problem Python Was Created to Solve

The software I have chosen, Python, is distributed under the Python Software Foundation (PSF) License. This is a permissive open-source license that gives users a lot of freedom to use, modify, and distribute the software however they need to.

The concept of free software is built around four main freedoms, originally defined by Richard Stallman. The first is the freedom to run the program for any purpose meaning no one can tell you what you are or aren't allowed to do with it. The second is the freedom to study how it works, which is only possible because the source code is open. The third is the freedom to modify the code and make it work the way you want. The fourth is the freedom to share copies with others, including your modified versions. Python's PSF License supports all four of these, which is part of why it spread so widely and so quickly across different fields.

One thing that surprised me when I was reading about this is that a company can actually take Python, modify it, and sell it as part of their own product without having to release their changes. The PSF License allows this. Companies like Anaconda have built commercial products directly on top of Python, and they are not legally required to give those modifications back to the community. This is what makes it a permissive license rather than a strict one.

This is where it gets interesting when you compare it to other licenses. The GPL, or General Public License, works differently if you distribute a modified version of GPL software, you must also release that modified version as open source. You cannot keep your changes private. The MIT License, on the other hand, is even more relaxed than the PSF License and basically just says "do whatever you want, just keep my name on it." The PSF License sits close to MIT in practice it allows flexibility and does not force anyone to share their modifications, but it does require the original copyright notice to stay intact.

There is also an important difference between "free as in free beer" and "free as in freedom," and Python is actually both. Free beer just means no cost you can download it without paying anything. Freedom means something deeper: you have the right to look inside it, change it, and share it. A company could theoretically release a tool for free to download but keep the source code completely hidden. That would be free beer but not freedom. Python is different because the source code is always available, always readable, and always modifiable. That is what the PSF License actually protects.

While studying this for my project, I realised that the choice of license is not just a legal decision it shapes how a community grows around a piece of software. Because Python placed almost no restrictions on what people could do with it, developers, researchers, and companies all jumped in without hesitation. That openness is a big reason Python ended up powering everything from university assignments to NASA pipelines.

A3. The ethics of open source

Open source software is not just about code it is also about how knowledge gets shared and who gets access to it. When I first started this project, I assumed open source was simply "better" in every situation. But after reading more about it, I think the answer is a bit more complicated than that.

Not all software should be open source, in my opinion. There are certain types of software like systems used in defence, healthcare, or banking where making the code

publicly available could create serious security risks. If a hospital's patient data system had its source code open for anyone to read, it might also be open for anyone to exploit. In those cases, keeping the code private makes sense, not because of greed, but because of safety. That said, for general tools, libraries, and anything that forms shared infrastructure like an operating system or a programming language I think openness should be the default.

The ethical question of whether companies should profit from open-source work is one I found genuinely uncomfortable to think about. On one hand, it is legal and it does drive adoption. Python is everywhere partly because companies could build on it without restrictions. On the other hand, there are situations where large, profitable corporations use open-source tools maintained by a handful of volunteers and contribute almost nothing back. The Log4j incident a few years ago showed exactly this a critical piece of software used by thousands of companies worldwide was being maintained by basically two people in their spare time. That does not feel right to me. I think it is acceptable to build a business on open-source software, but there is a responsibility that comes with it, even if it is not written into the license.

The phrase "standing on the shoulders of giants" means that every new idea is built on top of work that came before it. In software, almost nothing starts from zero. When I wrote my five shell scripts for this project, I was using Bash, which runs on Linux, which depends on decades of open-source work going back to the 1970s. I did not build any of that I just used it. Open source makes that kind of inheritance visible and accessible. If all of those tools had been proprietary, learning to do what I did in this project would have been either very expensive or simply not possible for someone like me.

From my perspective, open source encourages innovation rather than limiting it. It means a student in a small city can learn with the same tools that engineers at big tech companies use. That is something worth protecting. While working on this project, I realised how much of what I take for granted Python, Ubuntu, even the terminal I was typing in exists because thousands of people chose to share their work openly. That changed how I think about software, and honestly about knowledge in general.

Part B Linux Footprint (Unit 2)

In this project, I installed Python on a Linux system using Ubuntu 25.10 inside a UTM virtual machine on my Mac. I had never really thought about what happens behind the

scenes when you install a programming language I always just downloaded things and used them. This part of the project actually changed that for me.

The installation itself was straightforward. I used the APT package manager with a single command:

```
None
sudo apt install python3
```

What I found interesting is that this one command does more than just install Python. It also pulls in all the dependencies automatically — libraries and tools that Python needs to run properly. I did not have to figure out what those were or install them separately. That is one of the practical advantages of how open-source software is packaged on Linux systems.

After installation, I started exploring where Python actually lives on the filesystem. The main executable is at:

```
/usr/bin/python3
```

This is the file that runs every time you type `python3` in the terminal. When I checked it using `ls -ld` in my Script 3, I could see it was a symbolic link owned by root — meaning it points to the actual binary but is managed at the system level. Other important files are stored under:

```
/usr/lib/python3
```

This directory holds the standard library modules that Python depends on internally. When I audited it during my disk and permission script, I found it was owned by root with `drwxr-xr-x` permissions readable by everyone but only writable by root. That makes sense, because you do not want regular users accidentally modifying core system files.

One thing I learned is that Python is not a service. Tools like Apache or MySQL run in the background as services even when no one is actively using them. Python does not work that way it only runs when someone actually executes a script or types a command. It runs as whatever user is logged in at that moment, which in my case was

my ubuntu user inside the VM. This also means that a Python script inherits the permissions of whoever runs it, which is why running Python as root is generally discouraged in real deployments.

For updates, Python is managed through the same package manager used for everything else on the system:

```
None
sudo apt update
sudo apt upgrade
```

The first command refreshes the list of available packages, and the second actually installs any newer versions. This means security patches and improvements reach the system through a single, consistent process rather than needing to manually download anything.

While working on this section, I understood something I had not thought about before that software does not just "exist" on a computer. It has a specific location, specific permissions, a specific owner, and a specific way of getting updated. Running my shell scripts and actually seeing these directories and permission strings in the terminal made all of this feel real rather than theoretical.

Part C The FOSS Ecosystem

Python does not exist alone it is part of a much larger open-source ecosystem, and once you start looking at how everything connects, it is actually quite impressive how central Python has become to modern software development.

At the base level, Python depends on other open-source tools to function. CPython, which is the standard version of Python that most people use, is written in C and relies on the GNU C Library underneath. It also uses OpenSSL for anything involving encryption or secure connections, zlib for compression, and sqlite3 for lightweight database support. None of these are Python's own work they are separate open-source projects that Python wraps and makes accessible to Python developers. This is a good example of how open-source software stacks on top of itself, each layer depending on the one below.

On top of Python, an enormous ecosystem has been built. The package manager pip allows anyone to install libraries from PyPI, which currently hosts hundreds of thousands of packages. The ones that come up most often are NumPy and Pandas for data work, Matplotlib for visualisation, and Requests for making HTTP calls. These are not small side projects NumPy alone is a dependency for almost everything in scientific computing, and it has been downloaded billions of times.

The frameworks built on Python show how far it has reached. Django and Flask are used for web development Instagram was originally built on Django, which says something about how seriously it can be taken at scale. For machine learning and AI, TensorFlow (from Google) and PyTorch (from Meta) are both primarily Python-facing, meaning even though a lot of the heavy computation runs in C++ underneath, the developer-facing layer is Python. The AI tools that people use and talk about today largely run through Python, which is something I found genuinely surprising when I looked into it for this project.

Python also connects well with Linux server environments. While it is not part of the traditional LAMP stack (Linux, Apache, MySQL, PHP), Python can replace PHP in many setups and works naturally with Linux, Apache or Nginx, and databases like PostgreSQL or MySQL. Frameworks like Django come with their own built-in development server and database management tools, which makes the whole stack more self-contained.

The community around Python is one of the things that keeps it growing. The Python Software Foundation organises PyCon every year, which draws tens of thousands of developers globally. Day-to-day, the community lives on GitHub, where the CPython repository has thousands of open issues and pull requests being actively discussed. There are also official forums, a mailing list, and Stack Overflow, where Python consistently ranks as one of the most-asked-about languages. When I ran into questions while writing my shell scripts for this project, almost every answer I needed was already documented somewhere by someone in this community.

While working on this part of the project, I realised that Python's success is not just about the language itself being good. It is about the ecosystem that formed around it the libraries, the frameworks, the tools, the forums, and the people who keep contributing without being paid to do so. That combination is what makes it genuinely hard to replace.

PART D — Open Source vs Proprietary (Python vs MATLAB)

One of the best ways to understand the value of open-source software is to compare it directly with a proprietary alternative. For Python, the closest comparison in the scientific and data computing space is MATLAB a powerful tool developed by MathWorks that is widely used in engineering and research, but comes with significant restrictions.

Feature	Open Source (Python)	Proprietary (MATLAB)
Cost	Completely free to download and use	Requires a paid license, which can be expensive
Security	Source code is publicly available and can be reviewed by anyone	Source code is closed, so users have to trust the company
Support	Community support through GitHub, forums, Stack Overflow, and official docs	Official MathWorks support included with the license
Flexibility	High users can modify, extend, and customize freely	Limited to the features MathWorks provides
Control	Full control in the hands of the user	Controlled entirely by the company

Based on this comparison, Python has clear advantages for most developers, students, and researchers. It costs nothing to get started, which alone removes a major barrier especially for someone like me who is learning and experimenting rather than working inside a large institution with a software budget. The flexibility is also something I noticed practically while working on this project. Every time I needed a library or a tool, I could just install it with pip and move forward. There was no approval process, no license check, no waiting.

On the other hand, MATLAB does have areas where it holds up. The official support that comes with a paid license is more structured and predictable than community forums, which matters in professional environments where downtime is costly. There are also some highly specialised engineering toolboxes in MATLAB that do not have direct Python equivalents yet. For a large company working in a regulated industry, that kind of guaranteed reliability might be worth the cost. But for the vast majority of use cases

web development, data science, automation, machine learning, general scripting Python covers everything MATLAB does and more, for free.

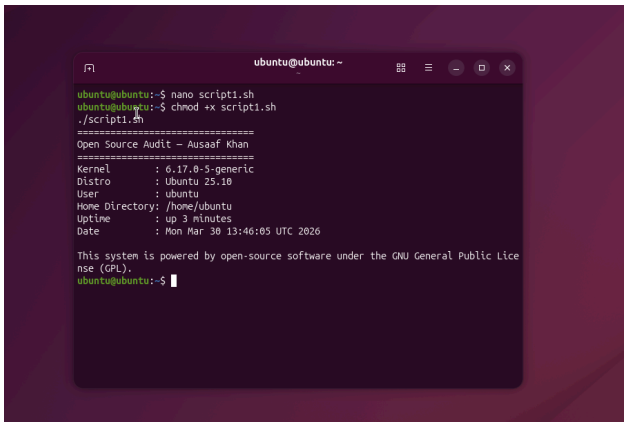
In my opinion, Python is the better choice for most real-world applications because of its flexibility, accessibility, and the strength of the ecosystem behind it. It does not lock you in, it does not expire, and it does not limit what you can build or learn. For students especially, that openness makes a real difference.

I would also be interested in contributing to open-source projects in the future. Working on this project made me see that open source is not something that just exists on its own it exists because people choose to put their time into it. Even something small, like reporting a bug or improving documentation, is a real contribution. After spending time reading through Python's GitHub and understanding how the community works, that kind of participation feels less intimidating than it did before. It is something I want to do more of.

Script 1 — System Identity Report

```
None
#!/bin/bash
# Script 1: System Identity Report
# Author: Ausaaf Khan

# --- Variables ---
STUDENT_NAME="Ausaaf Khan"
SOFTWARE_CHOICE="Python"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)
HOME_DIR=$HOME

# --- Display ---
echo "================================"
echo "Open Source Audit — $STUDENT_NAME"
echo "================================"
```

```
echo "Kernel        : $KERNEL"
echo "Distro        : $DISTRO"
echo "User          : $USER_NAME"
echo "Home Directory: $HOME_DIR"
echo "Uptime        : $UPTIME"
echo "Date          : $DATE"

echo ""
echo "This system is powered by open-source software under the GNU General
Public License (GPL)."
```

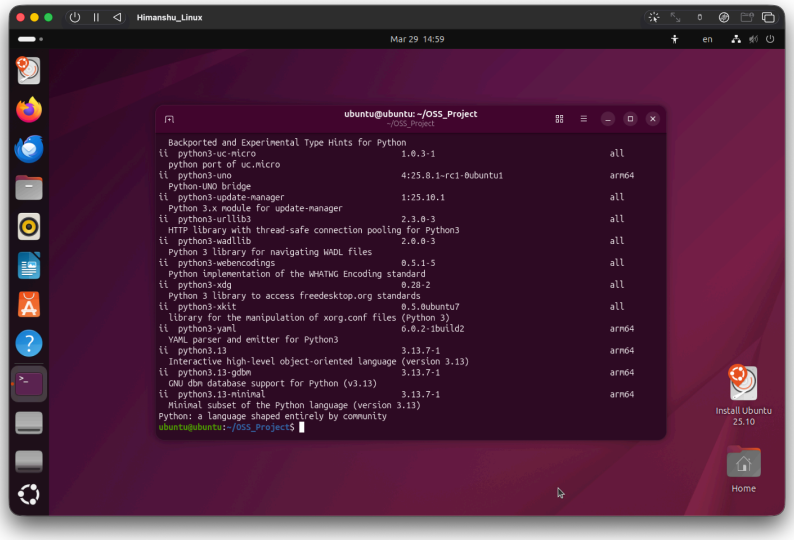


This script displays basic system information such as kernel version, user name, uptime, and Linux distribution. It uses commands like uname, whoami, and uptime to collect system data.

I used variables to store the values and echo statements to display them in a formatted way. This script helped me understand how to use command substitution and variables in shell scripting.

Script 2 — FOSS Package Inspector

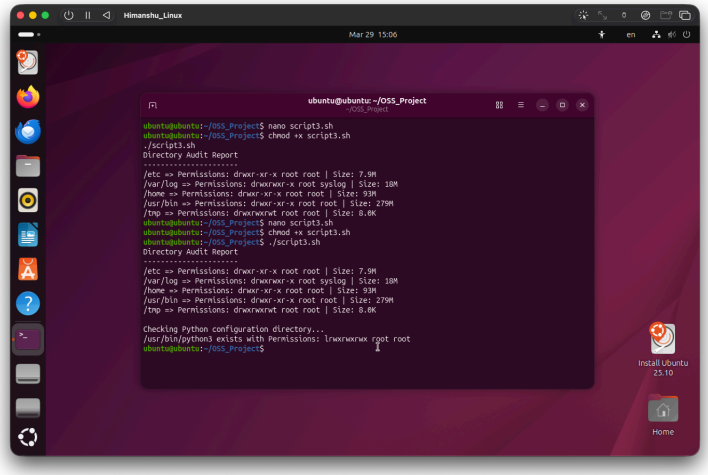
```
None
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="python3"

# Check if package is installed
if dpkg -l | grep -q "$PACKAGE"; then
    echo "$PACKAGE is installed."
    dpkg -l | grep "$PACKAGE"
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement
case $PACKAGE in
    httpd)
        echo "Apache: the web server that built the open internet"
        ;;
    mysql)
        echo "MySQL: open source at the heart of millions of apps"
        ;;
    vlc)
        echo "VLC: plays anything — built by students in Paris"
        ;;
    firefox)
        echo "Firefox: a nonprofit fighting for an open web"
        ;;
    python3)
        echo "Python: a language shaped entirely by community"
        ;;
esac
```



This script checks whether a package (Python) is installed or not. It uses dpkg and grep commands to verify installation.
The script also uses a case statement to print a short description based on the package name. This helped me understand conditional statements in shell scripting.

Script 3 — Disk and Permission Auditor

```
None
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo ""
echo "Checking Python configuration directory..."

CONFIG_DIR="/usr/bin/python3"

if [ -e "$CONFIG_DIR" ]; then
    PERMS=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
    echo "$CONFIG_DIR exists with Permissions: $PERMS"
else
    echo "$CONFIG_DIR does not exist"
fi
```



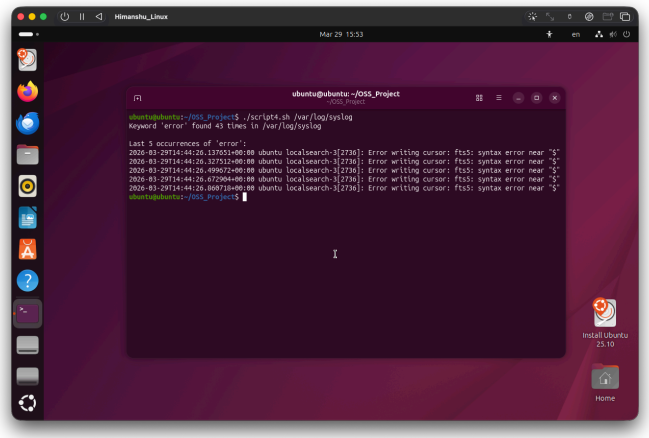
This script checks multiple system directories and displays their permissions and size. It uses a for loop to iterate through directories.

Commands like ls -ld and du are used to get permissions and disk usage. I also added a check for Python's directory. This script helped me understand loops and file handling in Linux.

Script 4 — Log File Analyzer

```
None
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
```

```
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 occurrences of '$KEYWORD':"
grep -i "$KEYWORD" "$LOGFILE" | tail -5

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "Log file is empty. Please try with another file."
fi
```



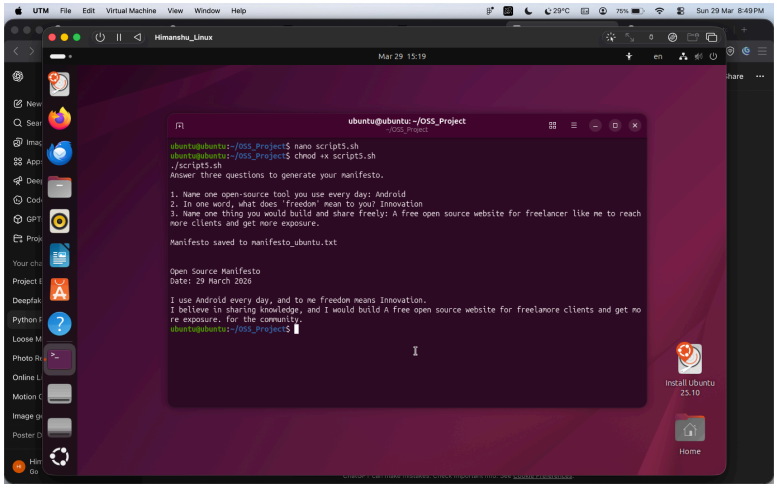
This script reads a log file line by line and counts how many times a keyword appears. It uses a while loop and if condition.

It also displays the last few matching lines using grep and tail. This helped me understand file reading and pattern matching.

Script 5 — The Open Source Manifesto Generator

```
None
#!/bin/bash
# Script 5: Open Source Manifesto Generator
```

```
echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "" > $OUTPUT
echo "Open Source Manifesto" >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "" >> $OUTPUT

echo "I use $TOOL every day, and to me freedom means $FREEDOM." >> $OUTPUT
echo "I believe in sharing knowledge, and I would build $BUILD for the
community." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```



This script takes user input and generates a manifesto file. It uses the read command to accept input and writes output to a file.
This helped me understand user interaction and file writing in shell scripting.